Algorithmic Architecture

Sorting, Searching, and Computational Optimizations for Offline UPI

1. Cryptographic Algorithms

Given the highly constrained bandwidth (160 GSM characters), symmetric and asymmetric cryptographic algorithms must minimize payload expansion while maintaining zero-trust security.

Elliptic Curve Digital Signature Algorithm (ECDSA)

Use Case: Hardware-level transaction validation.

Optimization: Compared to RSA, which requires at least 2048-bit keys (generating massive signatures), ECC achieves equivalent security with 256-bit keys. The resulting signature is compact enough (~64 bytes) to be included in a single SMS payload alongside transaction data.

Base85 (Ascii85) Compression Encoding

Use Case: Translating binary ciphertext to SMS-safe strings.

Optimization: Standard Base64 inflates binary data by ~33%. Base85 utilizes a larger character set, reducing the overhead to just ~25%. This 8% delta is often the difference between a successful single-part SMS and a fragmented multi-part SMS failure.

2. Searching Algorithms

The Server Tier must process incoming webhooks rapidly to prevent localized DDOS and ensure transactional integrity.

Replay Attack Prevention `O(1) Time`

Implementation: Hash Table (HashSet or Redis Cache).

Mechanism: Every incoming SMS payload contains a unique ***Txn_ID***. Before touching the SQL database, the Java Server queries a memory-resident Hash Map of recently processed IDs. If a collision occurs, the transaction is instantly dropped as a replay attack.

Ledger & Account Validation `O(log n) Time`

Implementation: B-Tree Indexing.

Mechanism: Queries against the PostgreSQL ledger (`SELECT balance FROM accounts WHERE phone = ?`) utilize B-Tree indexed searching. This prevents full table scans ***O(n)***, allowing the server to retrieve balances across millions of users logarithmically.

Contact Autocomplete (Client) `O(m) Time`

Implementation: Prefix Trie.

Mechanism: Searching local contacts by name/number on the iOS device utilizes a Trie data structure, where ***m*** is the length of the search string. This provides real-time autocomplete UI updates without UI-thread blocking.

3. Sorting & Queuing Algorithms

Offline states disrupt the chronological flow of data, requiring intelligent sorting mechanisms to re-establish the timeline.

Offline Promise Dispatch `O(n log n)`

Implementation: Priority Queue (Min-Heap).

Mechanism: When a user generates multiple offline transactions, they are added to a local Core Data queue. When cellular connectivity resumes, a Min-Heap structured by the `device_timestamp` ensures the oldest transaction is always popped and SMS-dispatched first.

Hash-Chain Resolution `O(V + E)`

Implementation: Topological Sort (Directed Acyclic Graph).

Mechanism: Telecom networks may deliver SMS messages out of order. Because each offline transaction contains the SHA-256 hash of the *previous* transaction, the server treats them as a DAG. A topological sort re-constructs the exact chronological order of the user's offline session before committing the balance deductions.